AI-Driven Network Traffic Analysis and Intrusion Detection System

Technical Report

Author: Praneeth Rayalu Repalle, B.Tech CSE (Cyber Security), Lovely Professional University, 2026

Prepared for: MegaMinds IT Services, Cybersecurity & Network Security Research Intern, Round 1 take-home

Submission: October 2026. Code, data, models and every figure are in the accompanying repository (AI-NIDS-Praneeth).

1. Problem overview

Networks face port scans, denial-of-service floods, credential brute force and man-in-the-middle attacks. Signature systems such as Snort are precise for patterns someone has already written down, but miss variants that sit just under a threshold. Pure machine-learning detectors can generalise, but they raise alerts an analyst cannot verify. The task is a working prototype that ingests traffic, extracts meaningful features, classifies traffic as normal or malicious, and gives the analyst enough evidence to act on each alert.

2. Objectives

1. Produce labelled traffic with benign activity and at least four attack types, reproducibly.
2. Extract flow- and time-window features that capture rate, protocol mix, connection behaviour and ARP integrity.
3. Detect attacks with a hybrid engine: readable threshold rules, a supervised classifier and an unsupervised anomaly model.
4. Explain every alert: what, why, confidence and severity.
5. Provide a CLI and a dashboard, and evaluate honestly, including a case the system gets wrong.

3. Traffic data

I generated the traffic with **scapy** rather than downloading CICIDS2017. CICIDS2017 is many gigabytes and has known labelling artefacts. My generator gives exact ground truth (attacker address, attack interval, class) and is fully seeded: regenerating with seed 42 gave byte-identical pcaps, which I checked with md5sum. The simulated network is a 192.168.10.0/24 LAN behind gateway 192.168.10.1, with a server at .50 (HTTP, HTTPS, SSH, FTP), a backup server at .60, workstations at .10–.29, and attackers inside the LAN and in 203.0.113.0/24.

Benign traffic mixes DNS lookups followed by HTTP or TLS-like HTTPS sessions with full handshakes and teardowns, intranet requests, ICMP echo, one long interactive SSH session and normal ARP. Captures are written like `tcpdump -s 96`: headers plus the first payload bytes, with the true length kept in each record. This keeps the five test pcaps at 19 MB.

Scenario	Content	Packets	Span
S1 Benign	15 workstations, mixed protocols, admin SSH, ARP	29,251	120 s
S2 Port scan	SYN scan of 999 ports at ~230 probes/s, then a slow 60-port scan at ~1.5 probes/s	27,264	120 s
S3 SYN flood	6 bots, ~1,300 SYN/s for 30 s on the web server; legitimate clients degraded	73,449	120 s
S4 Brute force + ARP	SSH brute force (225 tries), FTP brute force (60 tries, '530' replies), ARP poisoning of 3 hosts	30,293	153 s
S5 Ambiguous	Low-and-slow scan (1 probe per ~12 s) and a benign rsync backup at 12 connections/s	47,350	360 s

Train/test separation. The models are trained only on 12 separate training captures (3 per scenario S1–S4). These use different seeds, attacker addresses, scan sizes, flood rates and brute-force rates. Scenario 5 never appears in training. The training table has 6,634 host-windows: 6,532 benign, 51 DoS, 20 brute force, 19 port scan and 12 ARP spoofing.

4. System architecture

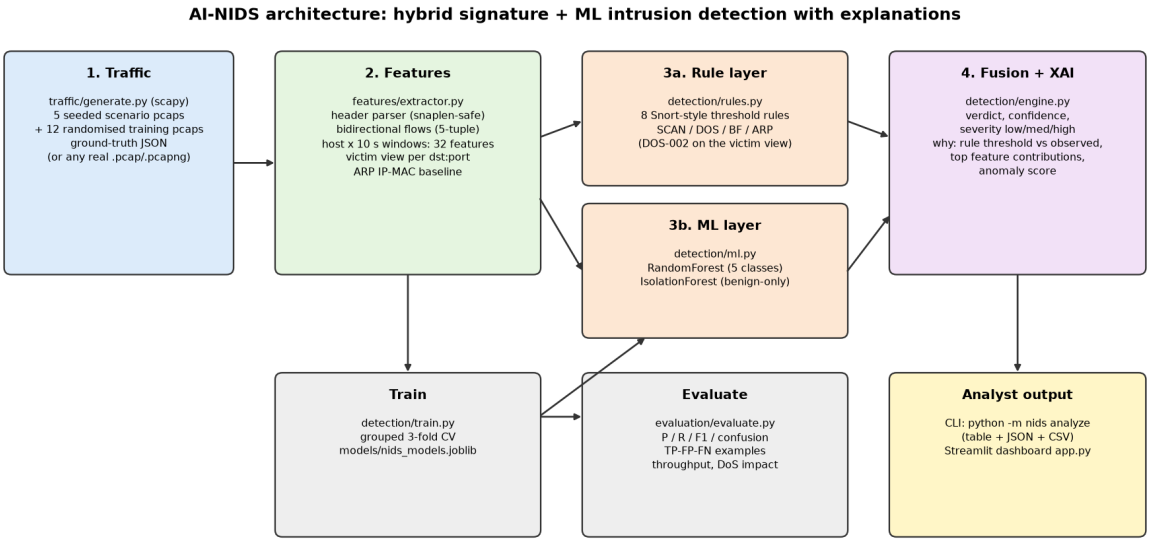


Figure 1. Pipeline: traffic, features, rule and ML layers, fusion with explanations, analyst outputs.

The code is split into `traffic/` (generator), `features/` (pcap reader, flow table, windows), `detection/` (rules, models, fusion, training), `evaluation/`, a CLI (`python -m nids analyze --pcap X`) that prints an alert table and writes JSON/CSV, and a Streamlit dashboard (`app.py`) for choosing or uploading a pcap, viewing alerts with their evidence, and viewing charts.

5. Feature extraction

A struct-based parser reads Ethernet, 802.1Q, IPv4, TCP, UDP, ICMP and ARP headers. It tolerates truncated captures and uses the IP total-length field for byte counts. Packets are assembled into **bidirectional flows** (5-tuple, 60 s idle timeout). Each flow tracks handshake state (SYN, SYN-ACK, completing ACK), rejection by RST, teardown, handshake latency, and login-failure replies (`530`, `Login incorrect`) for clear-text protocols.

The unit of classification is a **host-window**: everything one source did in one 10-second window. A single scan probe looks exactly like one failed connection. Grouping a source's behaviour over a window exposes scans, floods and brute force, and each flow statistic is kept as a mean or ratio. The 32 features fall into four groups:

- **Volume/rate:** packets, bytes, packet and byte rate, mean packet size.
- **Protocol distribution and flags:** TCP/UDP/ICMP/ARP fractions, SYN count, SYN-ACKs, RSTs, FINs, SYN ratio, SYN/ACK ratio.
- **Connection behaviour:** flows started and connection rate, unique destination IPs and **ports**, max connections to one service, mean flow duration/packets/bytes, short-flow share, handshake completion, rejected ratio, connections to authentication ports, login failures.
- **ARP integrity:** requests, replies, unsolicited replies and **IP-to-MAC binding changes**. These are judged against a baseline of which MAC actually sources each LAN IP's traffic, so a forged reply is caught even when it arrives before the genuine one. ARP frames are attributed to the sender MAC's host, which names the attacker rather than the impersonated gateway.

Two side views complement the windows: a **victim view** per destination:port and window, which catches floods with spoofed sources that no single source window shows, and a list of ARP binding changes for the analyst.

6. Detection methodology

6.1 Rule layer

Eight Snort-style threshold rules run in this layer: seven on every host-window and one (DOS-002) on the victim view. Each is written next to an equivalent Snort signature (for example `detection_filter: track by_src, count 20, seconds 10`):

Rule	Detects	Condition per 10 s window
SCAN-001	SYN port scan	unique dst ports >= 20 and rejected ratio >= 0.5
SCAN-002	Host sweep	unique dst IPs >= 20 and rejected ratio >= 0.5
DOS-001	SYN flood (source)	SYNs >= 100, handshake completion <= 0.2, >= 50 connections to one service
DOS-002	SYN flood (victim view)	>= 300 SYNs to one dst:port with completion <= 0.2, any number of sources
BF-001	Brute force	>= 10 connections to auth ports, >= 10 to one service, mean duration <= 5 s
BF-002	Clear-text login failures	>= 5 '530' / 'Login incorrect' replies
ARP-001	ARP spoofing	a known IP claimed by a different MAC
ARP-002	ARP poisoning	>= 5 unsolicited ARP replies

6.2 ML layer

A **RandomForest** (150 trees, depth 12, balanced class weights) classifies log1p-scaled windows into five classes. Trees handle mixed-scale counts and give feature importances for explanation. An **IsolationForest** (200 trees) is fitted on benign windows only and flags windows unlike normal traffic, which covers attacks no class describes. Its threshold is the 0.2nd percentile of benign training scores. A first version used the 1st percentile; on the test captures that produced 42 false positives on tiny harmless windows, so I tightened it using only the training score distribution. Grouped 3-fold cross-validation (whole captures held out) reached 0.9997 accuracy. That says the synthetic classes separate easily, not that the model will generalise to real traffic.

6.3 Fusion, confidence and severity

If a rule fires, the verdict is the rule's class. Confidence comes from how far the observed value exceeds the threshold (0.70 at the threshold, rising with the log of the ratio), combined noisy-OR with the RF probability when the RF agrees, and capped at 0.99. Otherwise, an RF attack class with p >= 0.5 gives an ML-only alert. Otherwise, an IsolationForest anomaly gives a low-severity **anomaly** for triage. DoS and ARP spoofing are high severity; scans and brute force are medium; a single-layer detection below 0.6 confidence drops one level; a brute force confirmed by both rules and RF rises to high.

6.4 Explainability

Each alert lists source, destinations, ports and window; each matched rule with observed value vs threshold; RF class probabilities; the top three features by RF importance x deviation from the benign median; and the IsolationForest score vs threshold. A real alert from Scenario 4:

```
EVT-0007 192.168.10.77 -> 192.168.10.50:21  10:01:10-10:01:20  bruteforce / high / 0.99
  BF-001: auth_service_conns=17 >= 10, max_conn_same_service=17 >= 10,
          mean_flow_duration=0.912 <= 5
  BF-002: auth_failures=17 >= 5
  RF: bruteforce p=1.00; top features: auth_service_conns=17 (benign median 0), ...
  IsolationForest score -0.162 (threshold -0.161) ANOMALOUS
```

7. Testing and evaluation method

Every host-window is labelled from the generator's ground truth: malicious if its source is a listed attacker and the window overlaps that attack's interval. Victim responses (for example the server's SYN-ACKs) count as

benign. I report precision, recall, F1, accuracy and confusion matrices per scenario and overall, per-layer results, attack-level detection with time to detect, true-positive, false-positive and false-negative examples, and throughput. Six pytest tests cover pcap round-trip, scan features and rules, ARP attribution, a benign window matching no rule, unique rule ids, and an end-to-end brute-force alert with explanation. All six pass.

8. Results

All numbers below are the actual output of `python -m nids evaluate` (unit: host-window, positive = malicious).

Scenario	Windows	Malicious	TP	FP	FN	Precision	Recall	F1
S1 Benign	566	0	0	0	0	1.000	n/a	n/a
S2 Port scan	515	6	6	1	0	0.857	1.000	0.923
S3 SYN flood	500	18	18	3	0	0.857	1.000	0.923
S4 Brute force + ARP	599	12	12	1	0	0.923	1.000	0.960
S5 Ambiguous	1,020	17	17	4	0	0.809	1.000	0.895
Overall	**3,200**	**53**	**53**	**9**	**0**	**0.855**	**1.000**	**0.922**

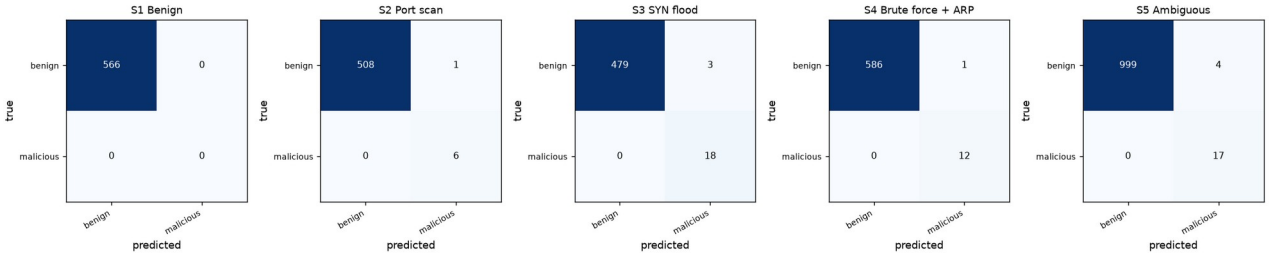


Figure 2. Binary confusion matrix per scenario (host-windows).

Layer	TP	FP	FN	Precision	Recall	F1
Rules only	31	0	22	1.000	0.585	0.738
RandomForest only	49	1	4	0.980	0.924	0.952
IsolationForest only	47	9	6	0.839	0.887	0.862
Hybrid (final)	53	9	0	0.855	1.000	0.922
Hybrid, medium+ severity only	35	1	18	0.972	0.660	0.786

Scenario 1: zero alerts on 566 benign windows. **Scenario 2:** the fast scan fired SCAN-001 (999 unique ports, 100% rejected, confidence 0.99). Three of the slow follow-up scan's windows stayed below the 20-port threshold and were caught only by the RandomForest; that is the main reason the hybrid beats rules alone. **Scenario 3:** all 18 bot windows fired DOS-001 and DOS-002 fired on the victim service. From the capture, legitimate clients' median SYN to SYN-ACK time rose from 1.0 ms to 173 ms and 39.2% of their SYNs went unanswered. The generator models the overloaded server, so this demonstrates the measurement, not a real server's behaviour. **Scenario 4:** SSH and FTP brute force fired BF-001 (and BF-002 for FTP), raised to high because rules and RF agree. ARP-001 named 192.168.10.99 (MAC 02:00:00:00:00:63) claiming the gateway's IP, with 108 binding changes listed.

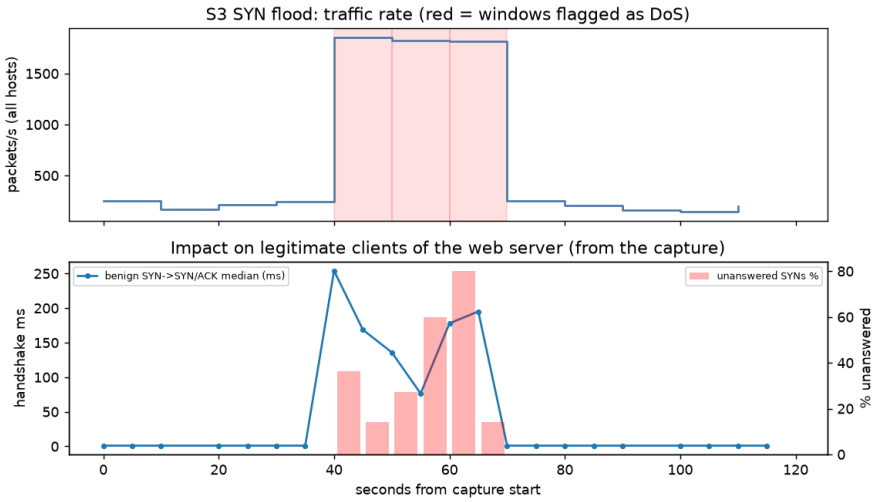


Figure 3. Scenario 3: traffic rate with DoS windows shaded, and impact on legitimate clients.

Scenario 5 (borderline): both halves expose real limits. Each window of the low-and-slow scan holds one probe. Thirteen were labelled portscan by the RF at p = 0.51, a coin flip, and four probes to open ports were only IsolationForest anomalies, so all 17 are low severity. If analysts are paged only on medium or high alerts, this scan is missed entirely (the medium+ row above). The same weak evidence, a single unanswered SYN, also produced the one medium false positive: a legitimate client in Scenario 3 whose SYNs the flooded server dropped (RF portscan, p = 0.69). The benign backup burst (up to 156 short, fully completed connections per window) matched no rule, but four of its windows were flagged as anomalies: false positives a real SOC would suppress with an allow-list for the backup window.

False-positive breakdown (9): four backup windows (S5); three hosts whose repeated pings were merged into one 60–120 s ICMP 'flow', an artefact of keying ICMP flows without the echo identifier (S2, S3, S4); and two legitimate clients with unanswered SYNs during the flood (S3). Eight of the nine are low severity. **False negatives:** none at the default operating point; 18 at medium+ (17 slow-scan windows and one slow-scan window in S2).

Throughput and latency: on a shared 4-vCPU cloud machine, extraction plus detection ran at 12,263–38,569 packets/s, 27–157 times faster than the captures' real duration, with 0.3–6.0 ms of detection per window. Alerting itself waits for the 10 s window to close, so time to detect was 10 s for most attacks.

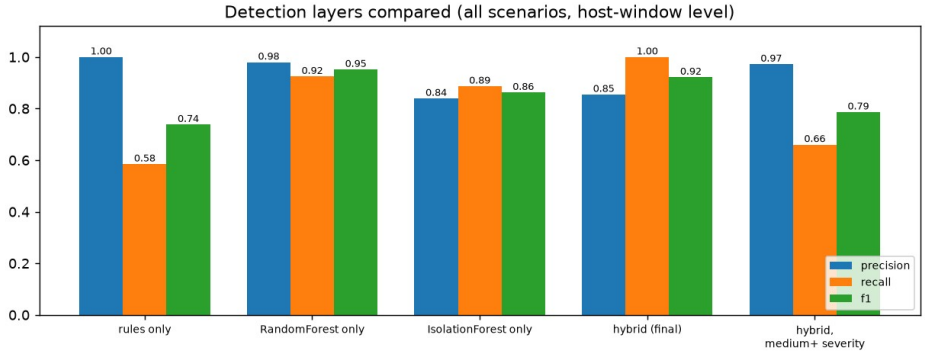


Figure 4. Precision, recall and F1 of each layer and of the fused verdict.

9. Network and security considerations

- **Latency and throughput:** offline Python processing at around 12–38k packets/s is enough for a small LAN or a SPAN port, but not inline on a busy gigabit link (about 80,000 packets/s with full-size frames, over a million with small ones). The engine should sit out of band on a mirror port, so a slow detector cannot drop traffic. Window size trades detection delay against evidence: 10 s windows mean up to 10 s before an alert.

- **Data completeness:** a snaplen of 96 bytes, mirror-port drops and asymmetric routing remove payload or half of a conversation. Handshake-completion features assume both directions are visible. On a one-way tap a normal client looks like a scanner.
- **Evasion:** slow scans (S5), scans spread across many sources, spoofed floods (handled only by the victim view), brute force paced under 10 attempts per window, and encrypted protocols (SSH is detected from behaviour, not content). An attacker who learns the thresholds can stay just under them; the ML layer helps only near the training distribution.
- **False-positive impact:** each alert costs analyst time, and low-confidence alerts erode trust. That is why IsolationForest-only alerts are low severity, the threshold is set as an alert budget, and severity is reduced when only one layer weakly agrees. Automatic blocking on a false DoS verdict could cut off a legitimate backup.
- **Scalability:** flow state grows with concurrent connections. At larger scale I would shard by source IP, use a compiled flow meter (Zeek or CICFlowMeter) feeding the same window features, stream windows through Kafka, and keep the models, which are small (a 0.6 MB joblib file) and fast at inference.

10. Limitations

- Synthetic traffic is cleaner and more regular than real networks, so the near-perfect recall is an upper bound.
- Small training set (19 scan, 51 DoS, 20 brute-force, 12 ARP windows), so the RandomForest partly memorises the generator.
- Fixed 10 s tumbling windows miss anything slower than one event per window.
- The ARP baseline is learned from the whole capture (offline); live use needs a trusted table (DHCP snooping or static entries).
- ICMP flows are keyed without the echo identifier, which caused three false positives.

11. Future work

- Multi-scale windows (10 s, 5 min, 1 h) with per-source state for low-and-slow attacks, plus correlation of windows into incidents.
- Evaluation on CICIDS2017 / UNSW-NB15 with Zeek features to test generalisation beyond my generator.
- SHAP explanations, calibrated probabilities, and an analyst feedback loop for allow-lists and retraining.
- Streaming capture (AF_PACKET / PF_RING) and export of the rule layer to real Snort/Suricata signatures.